Guía paso a paso – Laboratorio de Sistemas Distribuidos

Oct 6, 2026 · @Santiago

Cómo usar esta guía

Van a construir un **sistema de almacenamiento distribuido de archivos**: un coordinador y tres nodos en cuatro máquinas distintas, conectadas por Tailscale y comunicadas por HTTP/REST. El usuario sube y descarga archivos hablando solo con el coordinador, cada archivo queda guardado en 2 nodos y, si uno se cae, el archivo se sigue entregando desde el otro.

La guía va en el orden en que conviene trabajar. Cada fase termina con un **punto de control**: no pasen a la siguiente hasta cumplirlo, y tomen pantallazos en ese momento para la bitácora.

No trae el código resuelto. Describe qué debe hacer cada pieza, qué rutas exponer, la lógica en pseudocódigo y los detalles que suelen fallar. El código lo escriben ustedes en el lenguaje que dominen; se recomienda **Python con Flask o FastAPI** o **Node.js con Express**.

Cómo se relaciona con la guía del profe:

Requisito del profe	Dónde se cubre
Máquinas e IPs distintas, conectividad (secciones 3, 4, 8, 9)	Fase 1
Diseño y planeación (sección 7)	Fase 2
Un solo servicio para el usuario (sección 10)	Fases 4 y 5
Información real en los nodos (sección 11)	Fases 3 y 7
Distribución y replicación (sección 12)	Fase 4
Prueba obligatoria de fallo (sección 13)	Fases 4 y 7
Fallo del coordinador (sección 14)	Fase 8
Pregunta de los 100 nodos (sección 18)	Fase 8
Bitácora, video y entrega en Classroom	Fase 9

Fase 0: Organización del grupo

Antes de tocar código, definan quién hace qué. Lo más práctico es **una persona por máquina**, y que cada una sea responsable de explicar su parte en el video.

Rol	Máquina	Responsabilidad principal	Explica en el video
Integrante 1	Host A, coordinador	Programar el coordinador	Distribución, replicación y failover
Integrante 2	Host B, nodo 1	Programar el nodo (el mismo código sirve para los 3)	Almacenamiento y rutas del nodo
Integrante 3	Host C, nodo 2	Red con Tailscale, firewall, inventario	Arquitectura, red y conectividad
Integrante 4	Host D, nodo 3	Cliente o panel web, guion de pruebas, bitácora	Pruebas de fallo y análisis de 100 nodos

Si son menos de cuatro, alguien puede tener dos nodos en máquinas distintas (por ejemplo, su PC y su portátil). Lo que no vale es poner dos componentes en el mismo computador.

Pasos:

1. Creen un **repositorio en GitHub** con esta estructura y denle acceso a todos:

```
sistema-distribuido/
├── coordinador/
├── nodo/
├── cliente/        (si hacen panel web o CLI aparte)
├── docs/           (diagrama, tabla, pantallazos)
└── README.md
```

2. Creen un **documento compartido para la bitácora** (Google Docs, por ejemplo) con una sección por fase. Cada vez que alguien configure algo, encuentre un error o lo resuelva, pega el pantallazo con fecha y una línea de explicación.
3. Acuerden una **convención para los pantallazos**: que siempre se vea la terminal con el nombre del equipo y la IP. Así se demuestra que son máquinas distintas.
4. Confirmen que todos tienen instalado **Git** y el lenguaje elegido (Python 3.10 o superior, o Node.js 18 o superior).

Punto de control: repositorio creado, documento de bitácora creado, roles asignados.

Fase 1: Red entre las máquinas

El objetivo es que las cuatro máquinas se vean entre sí con una IP fija, estén donde estén. Esta fase se hace primero porque es la que más problemas da y no depende del código.

1.1 Instalar Tailscale

1. Una persona crea la cuenta en tailscale.com (con Google o GitHub) e invita a los demás a su red (tailnet) desde el panel de administración.
2. Cada integrante instala Tailscale en su máquina e inicia sesión aceptando la invitación.
3. Cada uno anota su IP con `tailscale ip -4`. Será del tipo `100.x.x.x` y no cambia aunque cambien de red.
4. En el panel web de Tailscale (Machines) deben aparecer las cuatro máquinas conectadas. Tomen pantallazo.

1.2 Tabla de inventario

Llenen la tabla de la sección 8 de la guía del profe. Los puertos que se proponen son 3000 para el coordinador y 4000 para los nodos.

Host	Responsable	Rol	IP Tailscale	Puerto	Estado
A		Coordinador	100.	3000	
B		Nodo 1	100.	4000	
C		Nodo 2	100.	4000	
D		Nodo 3	100.	4000	

1.3 Abrir el firewall

Windows bloquea el ping y las conexiones entrantes por defecto. En cada máquina Windows, en PowerShell como administrador:

```
New-NetFirewallRule -DisplayName "Ping" -Protocol ICMPv4 -IcmpType 8 -Action
Allow
New-NetFirewallRule -DisplayName "Lab SD" -Direction Inbound -Protocol TCP -
LocalPort 3000,4000 -Action Allow
```

En Linux con ufw: `sudo ufw allow 4000/tcp` en los nodos y `sudo ufw allow 3000/tcp` en el coordinador. En macOS normalmente no hace falta.

1.4 Probar conectividad

1. En cada máquina, muestren su identidad: `hostname` y `tailscale ip -4` (o `ipconfig` en Windows, `ip a` en Linux).
2. Desde el coordinador: `ping` a la IP de cada nodo.
3. Desde un nodo: `ping` al coordinador y a otro nodo.
4. Opcional pero muy útil: `tailscale status` muestra todas las máquinas de la red con su IP.

Recuerden lo que dice la guía: el ping demuestra conectividad, no un sistema distribuido. Eso viene después.

Punto de control: las cuatro máquinas aparecen en Tailscale, la tabla está llena y todos los pings responden. Pantallazos de cada paso en la bitácora.

Fase 2: Diseño antes de programar

El jueves 8 hay revisión de avances. Lleven este diseño escrito y la red funcionando. Es la sección 7 de la guía del profe y va completa en la bitácora.

2.1 Decisiones que deben documentar

Pregunta del profe	Decisión propuesta
Propósito	Almacenamiento distribuido de archivos: el usuario sube y descarga archivos sin saber en qué máquina quedan.
Arquitectura	Un coordinador (Host A) y tres nodos de almacenamiento (Hosts B, C, D).
Comunicación	HTTP/REST sobre la red de Tailscale. El usuario solo habla con el coordinador; el coordinador habla con los nodos.
Datos que viajan	El contenido de los archivos (binario), peticiones de estado (heartbeat) y listados de archivos en JSON.
Datos que se guardan	Cada nodo guarda archivos en su carpeta `storage/`. El coordinador guarda los metadatos en `metadata.json`: qué archivo está en qué nodos, tamaño y hash.

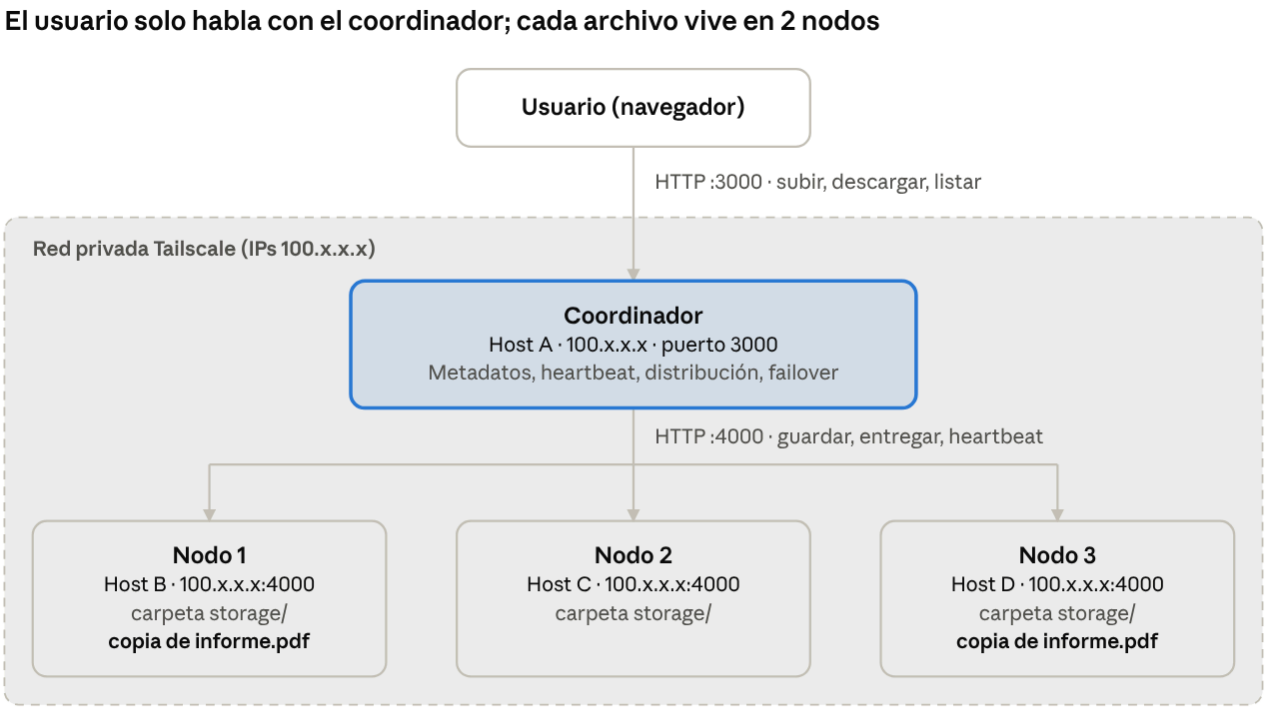
Pregunta del profe	Decisión propuesta
Distribución	Cada archivo nuevo va a los 2 nodos vivos con menos archivos (balanceo de carga).
Replicación	Factor 2: cada archivo existe en 2 nodos distintos. La copia se hace al subir, antes de responder al usuario (replicación síncrona).
Fallos de un nodo	El coordinador revisa cada 3 s qué nodos responden (heartbeat). Al descargar, si una réplica no responde en 2 s, pide el archivo a la otra.
Fallo del coordinador	Es un punto único de fallo. Mitigación: el coordinador puede reconstruir sus metadatos preguntando a los nodos qué tienen (ver Fase 8).

2.2 Responsabilidades de cada componente

- **Nodo:** guarda, entrega, lista y borra archivos de su carpeta, y responde si está vivo. No sabe que existen los otros nodos.
- **Coordinador:** es el único que el usuario conoce. Elige los nodos, replica, guarda los metadatos, vigila a los nodos y hace el cambio de réplica cuando una falla.
- **Cliente:** una página web servida por el coordinador o un script de consola. Solo conoce la IP del coordinador.

2.3 Diagrama de arquitectura

El profe pide un diagrama con hosts, IPs, puertos, protocolo y responsabilidades. El de abajo es la base; háganlo en draw.io o Excalidraw con sus IPs reales.



Si el Nodo 1 cae, el coordinador entrega informe.pdf desde el Nodo 3

arquitectura · coordinador y 3 nodos en 4 hosts

El usuario nunca habla con los nodos: el coordinador decide dónde guardar y, si una réplica no responde, pide el archivo a la otra.

Fase 3: Construir el nodo de almacenamiento

El nodo es la pieza más simple y la primera que deben tener funcionando: un servidor HTTP que guarda archivos en una carpeta. Es **el mismo programa en las tres máquinas**; lo único que cambia es su nombre.

3.1 Configuración por variables de entorno

No escriban el nombre del nodo ni el puerto dentro del código. Léanlos de variables de entorno, con valores por defecto:

Variable	Ejemplo	Para qué
`NODE_ID`	`nodo1`	Nombre del nodo, aparece en logs y respuestas
`PORT`	`4000`	Puerto donde escucha
`STORAGE_DIR`	`./storage`	Carpeta donde guarda los archivos

Así se lanza en cada máquina cambiando solo el nombre, y en desarrollo pueden correr tres nodos en un PC con puertos y carpetas distintas.

3.2 Rutas que debe exponer

Método	Ruta	Qué recibe	Qué responde
GET	`/health`	Nada	JSON con `node`, `status: "ok"`, `host` (nombre del equipo) y cantidad de archivos
PUT	`/files/<nombre>`	El archivo como cuerpo binario	201 y JSON con nombre, tamaño y sha256
GET	`/files/<nombre>`	Nada	El archivo, o 404 si no lo tiene
GET	`/files`	Nada	JSON con la lista de archivos que tiene en disco: nombre, tamaño y sha256
DELETE	`/files/<nombre>`	Nada	200 cuando lo borra

Enviar el archivo como cuerpo binario crudo (`application/octet-stream`) es más simple que un formulario multipart y funciona igual desde `curl` y desde el navegador.

3.3 Pasos de implementación

1. **Servidor básico.** Creen el servidor y pónganlo a escuchar en `0.0.0.0` y el puerto configurado. Si escucha en `127.0.0.1` o `localhost`, solo acepta conexiones de la misma máquina y no funcionará en red.
2. **Carpeta de almacenamiento.** Al arrancar, creen la carpeta si no existe.
3. **`/health`.** Respondan el JSON de estado. Incluyan el nombre del equipo (`socket.gethostname()` en Python, `os.hostname()` en Node): sirve para demostrar en qué máquina corre cada nodo.
4. **Guardar (`PUT`).** Lean el cuerpo completo y escríbanlo en `STORAGE_DIR/<nombre>`. Calculen el sha256 y devuélvanlo.
5. **Entregar (`GET`).** Si el archivo existe, devuélvanlo como binario; si no, 404.
6. **Listar (`GET /files`).** Recorran la carpeta y devuelvan nombre, tamaño y sha256 de cada archivo. Esta ruta la usará el coordinador para reconstruir sus metadatos.
7. **Borrar (`DELETE`).**

8. **Logs.** Cada operación imprime una línea con el nombre del nodo y la hora, por ejemplo `[NODO1 14:32:05] guardado informe.pdf (245 KB)`. En el video así se ve qué máquina trabajó.

Pseudocódigo del guardado:

```
PUT /files/<nombre>:
    nombre = solo_nombre_de_archivo(nombre)   # quitar rutas como ../../
    si nombre está vacío o empieza con "." → responder 400
    datos = leer cuerpo completo de la petición
    escribir datos en STORAGE_DIR/nombre
    log("guardado", nombre, tamaño)
    responder 201 {node, name, size, sha256(datos)}
```

Seguridad mínima: usen solo el nombre del archivo (`os.path.basename` en Python, `path.basename` en Node). Si no, alguien podría enviar `../../algo` y escribir fuera de la carpeta.

3.4 Ayudas según el lenguaje

- **Flask:** el cuerpo crudo se lee con `request.get_data()`; para devolver el archivo, `send_file(ruta)`. Ejecuten con `app.run(host="0.0.0.0", port=PORT)`.
- **FastAPI:** el cuerpo se lee con `await request.body()`; para devolver, `FileResponse(ruta)`. Ejecuten con `uvicorn main:app --host 0.0.0.0 --port 4000`.
- **Express:** para recibir binario usen `express.raw({ type: "*/*", limit: "100mb" })` en la ruta PUT; para devolver, `res.sendFile(rutaAbsoluta)`. Escuchen con `app.listen(PORT, "0.0.0.0")`.

3.5 Pruebas del nodo

Primero en su propio PC y luego desde otra máquina, cambiando `localhost` por la IP del nodo:

```
curl http://localhost:4000/health
curl -X PUT --data-binary @prueba.txt http://localhost:4000/files/prueba.txt
curl http://localhost:4000/files
curl http://localhost:4000/files/prueba.txt
curl -X DELETE http://localhost:4000/files/prueba.txt
```

Después de la subida, abran la carpeta `storage/` y confirmen que el archivo está ahí.

Punto de control: desde la máquina del coordinador, `curl http://IP_NODO:4000/health` responde para los tres nodos. Esto ya es comunicación entre procesos por red: pantallazo para la bitácora.

Fase 4: Construir el coordinador

El coordinador es lo que convierte tres servidores de archivos en un sistema distribuido: decide dónde va cada archivo, mantiene las réplicas y sigue respondiendo cuando un nodo se cae. Constrúyanlo por partes, probando cada una antes de seguir.

4.1 Configuración

La lista de nodos va en un archivo `nodes.json`, no en el código, para cambiar IPs sin tocar nada más:

```
[
  { "id": "nodo1", "url": "http://100.x.x.x:4000" },
  { "id": "nodo2", "url": "http://100.x.x.x:4000" },
  { "id": "nodo3", "url": "http://100.x.x.x:4000" }
]
```

Otros valores configurables: puerto (3000), réplicas por archivo (2), intervalo del heartbeat (3 s) y tiempo máximo de espera a un nodo (2 s).

4.2 Estado que guarda

- **Estado de los nodos** (en memoria): para cada nodo, si está vivo, cuándo respondió por última vez y su nombre de equipo.
- **Metadatos** (en `metadata.json`, para que sobrevivan a un reinicio): para cada archivo, su tamaño, su sha256, la fecha de subida y **la lista de nodos donde está**. Ejemplo: `"informe.pdf": { "size": 250000, "sha256": "9ee8…", "nodes": ["nodo1", "nodo3"] }`.
- **Eventos recientes** (en memoria, opcional): las últimas 50 líneas de log, para mostrarlas en el panel.

4.3 Rutas que debe exponer

Método	Ruta	Qué hace
PUT	`/api/files/<nombre>`	Recibe el archivo y lo guarda en 2 nodos
GET	`/api/files/<nombre>`	Devuelve el archivo desde alguna réplica disponible

Método	Ruta	Qué hace
DELETE	`/api/files/<nombre>`	Lo borra de todos sus nodos
GET	`/api/files`	Lista los archivos con sus nodos y si cada nodo está vivo
GET	`/api/status`	Estado de cada nodo y eventos recientes
POST	`/api/rebuild`	Reconstruye los metadatos preguntando a los nodos
GET	`/`	Panel web (Fase 5)

4.4 Paso 1: llamar a los nodos con tiempo límite

Todo el coordinador depende de una función que haga peticiones HTTP a un nodo **con timeout**. Sin timeout, cuando un nodo se cae la petición puede quedarse esperando 20 o 30 segundos y la demo se congela.

- **Python:** `requests.get(url, timeout=2)`; capturen `requests.exceptions.RequestException`.
- **Node.js 18+:** `fetch(url, { signal: AbortSignal.timeout(2000) })` dentro de un `try/catch`.

Para subir archivos usen un timeout más largo (unos 30 s), porque enviar el archivo toma tiempo.

4.5 Paso 2: heartbeat

Cada 3 segundos el coordinador consulta `/health` en cada nodo y actualiza su estado. Cuando un nodo cambia de vivo a caído, o al revés, lo registra en el log.

```
cada 3 segundos, para cada nodo (en paralelo si pueden):
    intentar GET nodo.url + "/health" con timeout 2 s
    vivo = (respondió con 200)
    si vivo es distinto a estado[nodo].vivo:
        log(nodo, "ACTIVO" si vivo, si no "CAÍDO")
    estado[nodo].vivo = vivo
    si vivo: estado[nodo].ultima_vez = ahora
```

En Python pueden usar un hilo (`threading.Thread`) con un ciclo y `time.sleep(3)`, o `asyncio` si usan FastAPI. En Node, `setInterval`.

Pruébenlo: arranquen el coordinador con los tres nodos encendidos, apaguen uno con Ctrl+C y vean aparecer «nodo2 CAÍDO» en unos segundos; enciéndanlo y debe aparecer «nodo2 ACTIVO».

4.6 Paso 3: elegir nodos (distribución)

```
elegir_nodos():
    candidatos = nodos que están vivos
    ordenar candidatos por cantidad de archivos que ya tienen (menos primero)
    devolver los primeros REPLICAS (2)
```

La cantidad de archivos por nodo se cuenta en los metadatos. Con esto los archivos se reparten solos entre los tres nodos. Alternativas que pueden mencionar: round-robin o un hash del nombre del archivo.

4.7 Paso 4: subir y replicar

```
PUT /api/files/<nombre>:
    si el nombre ya existe en metadatos → 409 (no sobrescribir)
    destinos = elegir_nodos()
    si no hay destinos → 503 "no hay nodos disponibles"
    datos = leer cuerpo completo
    para cada nodo en destinos:
        intentar PUT nodo.url + "/files/" + nombre con los datos
        si respondió bien → agregar nodo a guardados
    si guardados está vacío → 503
    metadatos[nombre] = {size, sha256, nodes: guardados, fecha}
    guardar metadata.json
    log(nombre, "guardado en", guardados)
    si len(guardados) < REPLICAS → log de advertencia
    responder 201 {name, nodes: guardados}
```

Decisión a explicar en la bitácora: la replicación es **síncrona** (se espera a que los dos nodos guarden antes de responder). Es más lenta pero más segura que responder primero y copiar después.

4.8 Paso 5: descargar con failover

Esta es la parte que se evalúa en la prueba de fallo.

```
GET /api/files/<nombre>:
    si no está en metadatos → 404
    orden = metadatos[nombre].nodes, con los vivos primero
    para cada nodo en orden:
        intentar GET nodo.url + "/files/" + nombre con timeout 2 s
        si respondió 200:
            log(nombre, "servido por", nodo, y cuáles fallaron antes)
            responder el archivo con encabezado X-Served-By: nodo
            terminar
        si falló:
            log(nodo, "no respondió, probando otra réplica")
            marcar nodo como caído
    log("ninguna réplica disponible")
    responder 503
```

El encabezado `X-Served-By` y la línea de log son la evidencia de qué nodo respondió. Prueben el caso difícil: apaguen un nodo y **descarguen inmediatamente**, antes de que el heartbeat lo detecte. El coordinador debe esperar máximo 2 s y pasar a la otra réplica.

4.9 Paso 6: listar, estado y borrar

- `/api/files` devuelve cada archivo con sus nodos y, para cada nodo, si está vivo. Así se ve qué archivos siguen disponibles.
- `/api/status` devuelve el estado de cada nodo (vivo, URL, nombre de equipo, cuántos archivos tiene) y los eventos recientes.
- `DELETE` borra el archivo en cada nodo que lo tenga y lo quita de los metadatos.

4.10 Paso 7: reconstruir metadatos desde los nodos

Si el coordinador pierde `metadata.json`, no sabría dónde está nada. Como cada nodo sabe qué tiene en disco, el coordinador puede preguntarles:

```
reconstruir():
    para cada nodo:
        intentar GET nodo.url + "/files"
        para cada archivo de la respuesta:
            si no está en metadatos → crearlo con size y sha256
            agregar este nodo a su lista de nodos (si no estaba)
    guardar metadata.json
```

Llámenla al arrancar el coordinador (después del primer heartbeat) y desde `POST /api/rebuild`. Esto les da una respuesta práctica para la pregunta del coordinador caído.

Punto de control: con los nodos en las máquinas reales y el coordinador corriendo, suben 3 o 4 archivos con `curl -X PUT --data-binary @archivo http://IP_COORD:3000/api/files/archivo` y cada uno queda en 2 nodos distintos. Apagan un nodo y la descarga sigue funcionando.

Fase 5: Cliente para el usuario

El usuario debe percibir un solo servicio (sección 10 de la guía): sube y descarga sin saber en qué nodo queda el archivo. Hay dos opciones; la página web es la que mejor se ve en el video.

Opción A: página web servida por el coordinador (recomendada)

Un solo archivo `index.html` con HTML y JavaScript, que el coordinador devuelve en la ruta `/`. Así se abre desde cualquier máquina en `http://IP_COORD:3000`. Debe tener:

1. **Estado de los nodos:** una tarjeta o columna por nodo, en verde si está vivo y en rojo si está caído, con su IP y nombre de equipo.
2. **Subir archivo:** un `<input type="file">` y un botón que haga `fetch("/api/files/" + encodeURIComponent(nombre), { method: "PUT", body: archivo })` y muestre en qué nodos quedó.
3. **Tabla de archivos:** una fila por archivo y una columna por nodo, marcando dónde está cada copia. Cuando un nodo cae, sus celdas se ven en rojo. Esta tabla responde de un vistazo a la pregunta «¿dónde está físicamente el archivo?».
4. **Descargar:** un botón por archivo. Después de descargar, muestren el valor del encabezado `X-Served-By`, por ejemplo «Lo entregó nodo3».
5. **Eventos:** la lista de eventos recientes del coordinador (nodos caídos, archivos guardados, failover).
6. **Actualización automática:** consultar `/api/status` y `/api/files` cada 2 segundos con `setInterval`.

Para descargar con `fetch` y además leer el encabezado: pidan el archivo, conviértanlo con `await respuesta.blob()`, creen un enlace con `URL.createObjectURL(blob)` y hagan clic en él desde el código.

Opción B: cliente de consola

Un script con comandos `subir`, `descargar`, `listar` y `estado` que llame a la API del coordinador e imprima qué nodo respondió. Es más rápido de hacer, pero en el video se lucen menos los cambios de estado.

En cualquier caso, el cliente **solo conoce la IP del coordinador**, nunca las de los nodos.

Punto de control: desde una máquina distinta al coordinador, abren el panel, suben un archivo, ven en qué nodos quedó y lo descargan.

Fase 6: Despliegue en las cuatro máquinas

Desarrollar todo en un solo PC está bien (tres nodos con puertos 4001, 4002 y 4003 y carpetas distintas), pero **eso no se entrega**. Cuando funcione en local, pasen a las máquinas reales cambiando solo `nodes.json`.

1. **Bajar el código en cada máquina:** `git clone` del repositorio (o `git pull` si ya lo tienen).
2. **Instalar dependencias** en cada máquina: `pip install flask requests` (o lo que usen) o `npm install`.
3. **Arrancar los nodos.** En cada host B, C y D:

   ```
   # Linux o macOS
   NODE_ID=nodo1 PORT=4000 python nodo.py
   # Windows PowerShell
   $env:NODE_ID="nodo1"; $env:PORT="4000"; python nodo.py
   ```

   Cambien `nodo1` por `nodo2` y `nodo3` en las otras máquinas (y `python nodo.py` por `node nodo.js` si usan Node).
4. **Verificar los nodos desde el coordinador:** `curl http://IP_NODO:4000/health` para los tres.
5. **Configurar el coordinador:** poner las IPs de Tailscale reales en `nodes.json`.
6. **Arrancar el coordinador** en el host A y abrir `http://IP_COORD:3000` desde otra máquina.
7. **Evitar que las máquinas se suspendan** durante las pruebas y la grabación: desactiven la suspensión automática y conecten los portátiles a la corriente.

Docker es opcional. Si no lo dominan, no lo usen: agrega configuración de puertos y redes sin sumar a lo que se evalúa.

Punto de control: los cuatro procesos corren en cuatro máquinas distintas, el panel muestra los tres nodos activos y en cada terminal se ven los logs de su componente. Pantallazo de las cuatro pantallas para la bitácora.

Fase 7: Pruebas obligatorias

Este guion sigue la secuencia mínima de la sección 16 de la guía. Háganlo completo una vez para la bitácora (con pantallazos de cada paso) y otra vez grabando el video.

1. **Máquinas e IPs diferentes.** En cada host: `hostname` y `tailscale ip -4`.
2. **Comunicación por red.** `ping` del coordinador a cada nodo y `curl http://IP_NODO:4000/health`.
3. **Servicio distribuido.** Suban 3 o 4 archivos desde el panel, desde una máquina que no sea el coordinador. El panel y el log del coordinador muestran en qué nodos quedó cada uno.
4. **Información en nodos reales.** Elijan un archivo y, en **cada host que tiene la copia**, muestren la carpeta `storage/` con el archivo y saquen su hash:
   - Linux o macOS: `sha256sum archivo.pdf`
   - Windows: `Get-FileHash archivo.pdf`

   Los dos hashes deben coincidir entre sí y con el que muestra el panel. Eso prueba que la réplica es idéntica, no un mensaje que dice «replicado».
5. **Replicación y distribución.** Muestren que los archivos quedaron repartidos entre los tres nodos y que cada uno está en dos.
6. **Caída de un nodo.** Detengan con Ctrl+C un nodo que tenga la copia del archivo elegido. En el panel su columna pasa a rojo y el log dice «CAÍDO».
7. **Recuperación desde otro nodo.** Descarguen el archivo: el panel dice qué nodo lo entregó y el log del coordinador muestra el failover. Repitan descargando **inmediatamente** después de apagar el nodo, para mostrar que el coordinador espera máximo 2 s y cambia de réplica.
8. **Regreso del nodo.** Vuelvan a arrancarlo: el heartbeat lo marca ACTIVO.
9. **Caída de las dos réplicas (opcional).** Si apagan ambos nodos que tienen un archivo, el sistema responde que no hay réplica disponible. Sirve para explicar el límite del factor de replicación 2.
10. **Caída del coordinador.** Detengan el coordinador con los nodos encendidos y sigan el análisis de la Fase 8.

En la evaluación en vivo el profe puede pedir apagar **cualquier** nodo al azar. Practiquen con cada uno.

Punto de control: los 10 pasos funcionan y hay pantallazos de cada uno.

Fase 8: Análisis técnico

Estas preguntas van en la bitácora y el profe las hará en vivo. Las respuestas de abajo son la base; escríbanlas con sus palabras y con lo que observaron en sus pruebas.

8.1 ¿Qué pasa si se cae el coordinador?

Prueba: detengan el coordinador con los tres nodos encendidos.

Qué sigue funcionando	Qué deja de funcionar
Los nodos siguen vivos y con sus archivos (`curl http://IP_NODO:4000/files` lo demuestra)	El usuario no puede subir, descargar ni listar, porque solo conoce al coordinador
Los archivos no se pierden: siguen en disco en cada nodo	Nadie vigila a los nodos ni hace failover
Se podría pedir un archivo directo a un nodo si se sabe dónde está	Sin el coordinador, el usuario no sabe en qué nodo está cada archivo

Conclusión: el coordinador es un **punto único de fallo** (Single Point of Failure). Los datos sobreviven, pero el servicio no.

Mitigación que ya implementaron: borren `metadata.json`, vuelvan a arrancar el coordinador y muestren que reconstruye el mapa de archivos preguntando a los nodos. Eso demuestra que el estado real está en los nodos.

Alternativas que pueden proponer:

- **Coordinador de respaldo (standby):** una segunda máquina con una copia de los metadatos que toma el control si el principal deja de responder.
- **Elección de líder:** los nodos eligen a uno como coordinador cuando el actual cae (algoritmos Bully o Raft).
- **Sin coordinador:** el cliente calcula con un hash del nombre en qué nodos debe estar el archivo y les pregunta directamente.

8.2 ¿Qué cambiaría con 100 nodos?

- **Distribución:** «el nodo con menos archivos» obliga a revisar todos los nodos. Con *consistent hashing*, el nodo de un archivo se calcula del nombre, y agregar o quitar un nodo solo mueve una parte pequeña de los archivos.
- **Cuello de botella:** hoy todos los archivos pasan por el coordinador. Con 100 nodos, el coordinador debería responder solo «está en el nodo 17 y el 42» y el cliente descargaría directo del nodo.
- **Heartbeat:** consultar 100 nodos cada 3 s desde un solo punto genera mucho tráfico. Se usan protocolos *gossip*, donde los nodos se vigilan entre ellos.
- **Fallos más frecuentes:** con 100 máquinas siempre habrá alguna caída. Hace falta **re-replicación automática**: cuando un nodo muere, copiar sus archivos desde las réplicas vivas a otros nodos hasta volver a tener 2 copias (o 3).
- **Metadatos:** también deben replicarse o distribuirse, porque un solo `metadata.json` no escala ni es tolerante a fallos.

8.3 Pregunta central de la sustentación

«¿Por qué es un sistema distribuido y no varios programas en computadores diferentes?» Respondan con lo que mostraron:

1. Los componentes corren en máquinas independientes con IPs distintas y cooperan por red para prestar **un solo servicio**.
2. El usuario no sabe ni necesita saber dónde están los archivos: el coordinador decide.
3. La información está **físicamente repartida** y replicada en varios nodos, verificable con el hash.
4. El sistema **tolera la caída de un nodo**: sigue entregando el archivo desde otra réplica.
5. Conocen sus límites: el coordinador es un punto único de fallo y saben cómo mitigarlo.

8.4 Ventajas y limitaciones para la bitácora

- **Ventajas:** diseño simple de explicar y depurar; tolera la caída de un nodo; los datos sobreviven a la caída del coordinador.
- **Limitaciones:** coordinador como punto único de fallo; los archivos pasan por el coordinador; sin re-replicación automática; los archivos se cargan completos en memoria, lo que no sirve para archivos muy grandes.

Fase 9: Bitácora, video y entrega

9.1 Bitácora en PDF

Si fueron pegando pantallazos desde la Fase 0, aquí solo se ordena. Estructura sugerida, que cubre todo lo que pidió el profe:

1. Portada: integrantes, curso, fecha.
2. Descripción del problema y del servicio.
3. Diseño y arquitectura: tabla de decisiones (Fase 2) y diagrama con IPs, puertos y protocolo.
4. Máquinas y nodos: tabla de infraestructura, sistema operativo de cada una.
5. Herramientas y tecnologías, con la justificación de cada una.
6. Configuración de cada máquina: Tailscale, firewall, instalación del lenguaje.
7. Desarrollo paso a paso: nodo, coordinador, cliente, con fragmentos de código explicados.
8. Pruebas de conectividad y comunicación (pings y `curl`).
9. Distribución y replicación (panel, carpetas `storage/` y hashes).
10. Prueba de caída de un nodo y recuperación desde la réplica.

11. Análisis de la caída del coordinador y de los 100 nodos.
12. Dificultades, errores y soluciones aplicadas.
13. Conclusiones del grupo.

Reglas para los pantallazos:

- Incluyan los **errores** y sus correcciones, no solo el resultado final. El profe lo pide explícitamente.
- En cada captura de terminal debe verse el nombre del equipo o la IP.
- Debajo de cada pantallazo, una línea que diga qué muestra y en qué máquina.

9.2 Video de sustentación

Todos los integrantes deben participar y cada uno explicar una parte. Guion de unos 10 a 15 minutos:

Parte	Quién	Qué mostrar
Presentación y problema	Integrante 1	Qué servicio construyeron y por qué es distribuido
Arquitectura y red	Integrante 3	Diagrama, tabla de IPs, Tailscale, pings
Nodo	Integrante 2	Código del nodo, rutas, carpeta `storage/`
Coordinador	Integrante 1	Heartbeat, elección de nodos, replicación, failover en el código
Demo en vivo	Todos	Subir, verificar hashes en los nodos, apagar un nodo, descargar desde la réplica
Coordinador caído y 100 nodos	Integrante 4	Prueba, análisis y propuestas
Conclusiones	Todos	La respuesta a la pregunta central

Grábenlo con OBS o la grabadora de pantalla, con las terminales de varias máquinas visibles (compartiendo pantalla en una videollamada, o grabando cada pantalla y editando). Hagan un ensayo antes de la grabación final.

9.3 Lista de entrega en Classroom

- [ ] Desarrollo del taller (documento de diseño y análisis)
- [ ] Código fuente organizado por componentes, con README de cómo ejecutarlo

- [ ] Bitácora completa en PDF
- [ ] Pantallazos y evidencias del proceso
- [ ] Video de sustentación con todos los integrantes
- [ ] Enlaces adicionales (repositorio, video si lo suben a YouTube o Drive)

Revisen la lista de evidencias de la sección 15 de la guía antes de entregar:

- [ ] Diagrama de arquitectura con hosts, nodos, IP, puertos, protocolo y responsabilidades
- [ ] Descripción del problema y del servicio distribuido
- [ ] Tabla de infraestructura
- [ ] Evidencia de conectividad entre hosts
- [ ] Evidencia de comunicación entre procesos
- [ ] Evidencia de información almacenada físicamente en los nodos
- [ ] Evidencia de distribución y replicación
- [ ] Evidencia de caída de un nodo y recuperación desde otra réplica
- [ ] Análisis técnico de decisiones, resultados, ventajas, limitaciones y fallos

Errores comunes y cómo resolverlos

Cuando les pase alguno, tomen pantallazo del error y de la solución: va en la sección de dificultades de la bitácora.

Síntoma	Causa probable	Solución
El ping no responde	Firewall de Windows bloquea ICMP, o la máquina no está conectada a Tailscale	Regla de firewall de la Fase 1; revisar `tailscale status`
El ping responde pero `curl` al puerto no	El servidor escucha en `127.0.0.1`, o el firewall no tiene el puerto abierto	Escuchar en `0.0.0.0`; abrir el puerto 3000 o 4000
`Connection refused`	El proceso no está corriendo o usa otro puerto	Revisar la terminal del nodo y el puerto en `nodes.json`
La demo se congela unos 30 s al caer un nodo	Las peticiones a los nodos no tienen timeout	Timeout de 2 s en cada llamada (Fase 4.4)
El archivo llega vacío o dañado	Se envió como formulario y se leyó como texto, o con codificación	Enviar binario crudo (`--data-binary`) y leer el cuerpo como bytes

Síntoma	Causa probable	Solución
Error al subir archivos grandes	Límite de tamaño del cuerpo en el framework	Subir el límite (en Express, `limit: "100mb"`; en Flask, `MAX_CONTENT_LENGTH`)
Nombres con espacios o tildes fallan	La URL no está codificada	Usar `encodeURIComponent` en el cliente y decodificar en el servidor
El coordinador no ve un nodo que sí está encendido	IP equivocada en `nodes.json` o el nodo cambió de red sin Tailscale	Usar siempre la IP de Tailscale y verificar con `curl`
Un nodo aparece caído durante la grabación	El portátil se suspendió o perdió wifi	Desactivar suspensión, conectar a la corriente
Funciona en local pero no entre máquinas	Las URLs de los nodos dicen `localhost`	Poner las IPs reales en `nodes.json`

Cronograma

La entrega es el martes 13 de octubre y el parcial el jueves 15. Dejen el lunes solo para la bitácora y el video, no para programar.

Día	Fases	Resultado esperado
Martes 6 y miércoles 7	0, 1 y 2	Roles, repositorio, Tailscale funcionando, tabla de IPs, pings y diseño escrito
Jueves 8	Revisión en clase	Mostrar diseño y conectividad al profe y resolver dudas
Viernes 9	3	Nodo terminado y probado desde otra máquina
Sábado 10	4 y 5	Coordinador con replicación y failover, panel web
Domingo 11	6, 7 y 8	Despliegue en las 4 máquinas, guion de pruebas completo, análisis, grabación del video
Lunes 12	9	Bitácora en PDF, README, revisión de la lista de entrega
Martes 13	Entrega	Subir todo a Classroom con tiempo

Hacer bien el laboratorio también es estudiar para el parcial: replicación, tolerancia a fallos y punto único de fallo son temas probables.